FP 10 (Haskell 6): Practical prep (tree traversal)

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

A note on class engagement

I am very worried about the class for the upcoming test and prac.

They have been set at a **fair** level, and they **will not be difficult if you engage and practise. You will certainly fail them if you don't practise Haskell.**

However, **most students are not doing the exercises every day.** Most do not attend lectures. Less than 300 of the 429 students have even opened the first lecture PDF.

You need to start taking the course seriously. As nice as you are, I don't want to see you again next year.

Function composition (2)

In our first lecture, I said that Haskell functions are first-class. We can use them as values and we can pass them around. We've seen that with map, filter, fold, etc…

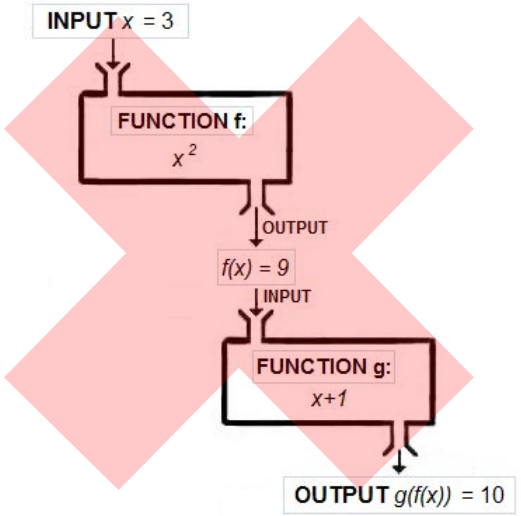
I also said that Haskell is expressive. This is both true in types and how we define functions.

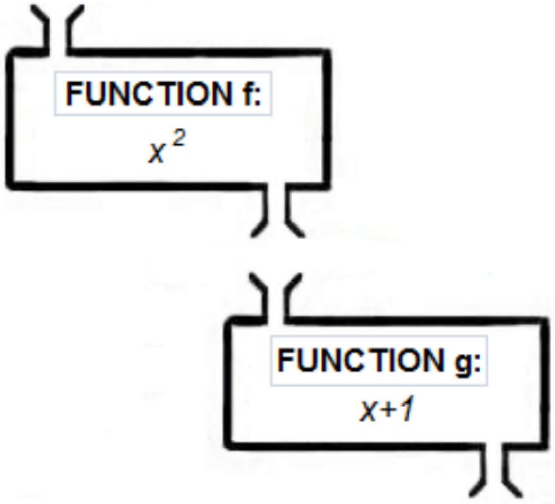
Another cornerstone of this is **function composition.**

Function composition allows us to define functions *just as combinations* of other functions - not even mentioning the data. This enables us to think at a higher level, about the *mapping* we want to use, rather than how our data is shuffled around.

Function composition

We sometimes think about functions (esp. in maths) as little **machines** which transform input values into output values. But, Haskell encourages us to try to think of the **functions themselves** rather than the values going in and out, through function composition.



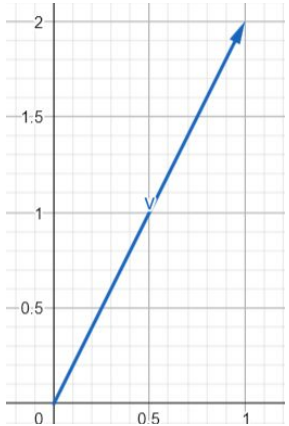


How do we use function composition?

Let's look at a simple example: the magnitude of a vector

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

So if our vector is (1, 2), we are calculating the **length** of that vector.



It's the same as the distance from (1, 2) to (0, 0).

Let's define our signature: `magnitude :: [Float] -> Float`

We can implement it like this:

```
magnitude xs = sqrt $ sum $ map (^2) xs
```

Or as a composition of functions like this:

```
magnitude = sqrt . sum . map (^2)
```

The . operator

The . operator is called “compose” or the “function composition operator” and is defined as follows:

```
f . g = \x -> f (g x)
```

So we have

```
(f . g) x = f (g x)
```

It’s *possible* to use . to compose functions with multiple arguments, but this is not as trivial. In fact, I gave up on a well-intentioned slide to teach it to you all before I was corrected by someone online.

The code in question was `flip flip 2 . ((^) .) . (-)`. If anyone can explain *that* to me right now (how it **works,** not what it does), I’ll give 5 points to the class… personally I am beat

Intro

We’ve got the class dog here with us to help us with the prac prep!

Oh no!!!





Note

We've received the following note from the evil wizard:



I HaVe tAKEn your DOg ANd LOckEd hEr in My dUnGEOn maze. MUAhaHAHA!
I am an Evil WizArd. But i am also a FAIr WiZARD.

If YoU CAn SeNd ME A HASkell proGRam To GUIdE yOuR doG tHRoUgH ThE DuNgeON, SHe can EscaPe aNd I WILl REtuRN her To you...

yoU HavE ONe wEEk...
Good luCk... muahaHa!!!

RANSOMIZER.COM

Note

We’ve received the following note from the evil wizard:



I have taken your dog and locked her in my dungeon maze. Muahahaha!
I am an evil wizard. But I am also a fair wizard.

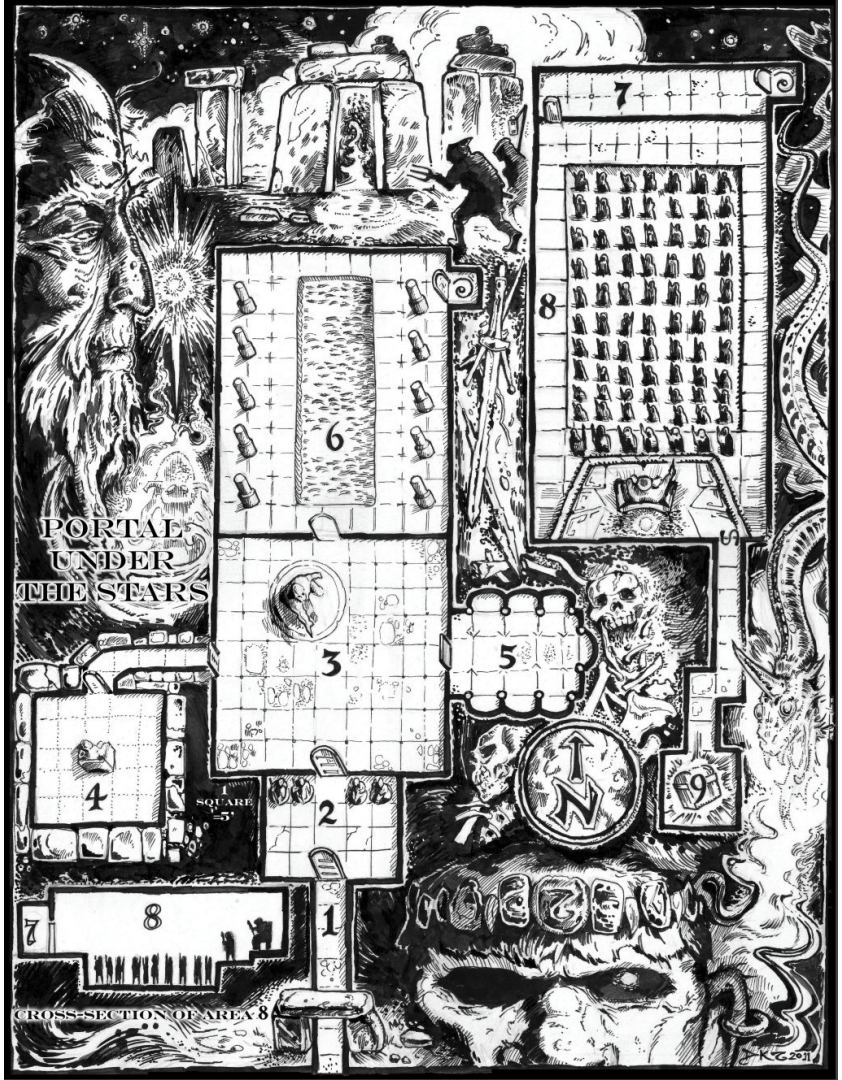
If you can send me a Haskell program to guide your dog through the dungeon, she can escape and I will return her to you...

You have one week...
good luck... muahaha!!!

Problem

The class dog is locked in the wizard's dungeon. A dungeon is like a maze, but it has **rooms** which are filled with **hazards**.

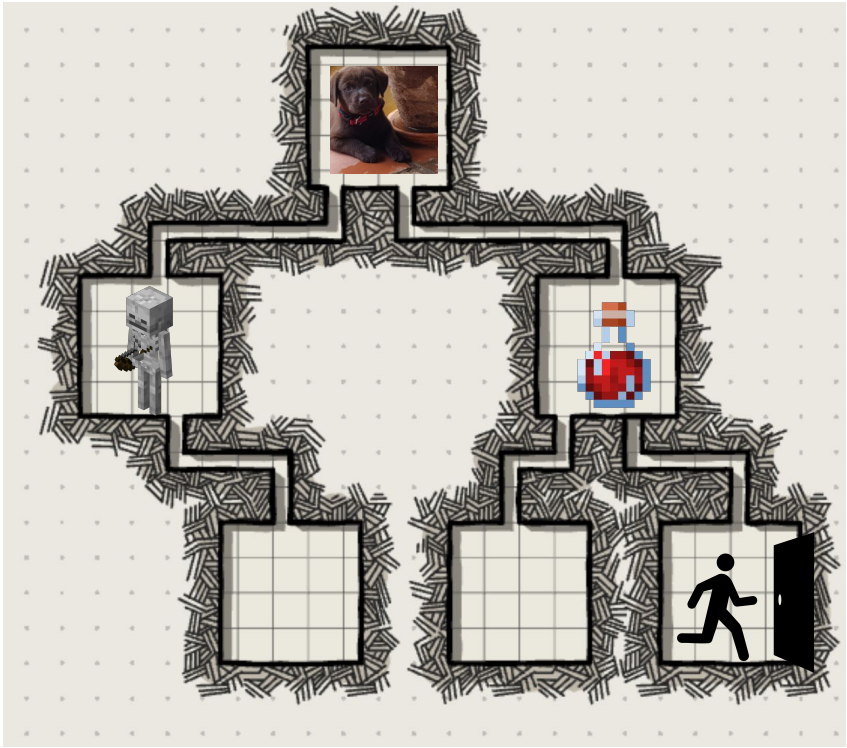
Any tabletop roleplaying game players should know all about this (D&D, Pathfinder, Dungeon Crawl Classics, etc…). The example on the right is of a dungeon from DCC's *Portal Under the Stars* booklet.



Refining our problem statement

The wizard has promised us that

- **each room has between 0 and 2 rooms it connects to** (these are called “child rooms”)
  - ... so, this is a **graph**
- **there are no cycles in the dungeon** - each room is only reachable from one path
  - … so, this is an ***acyclic* graph**
- **every room is reachable from the start room** - there are no ‘hidden rooms’
  - ... so, this is a **tree**

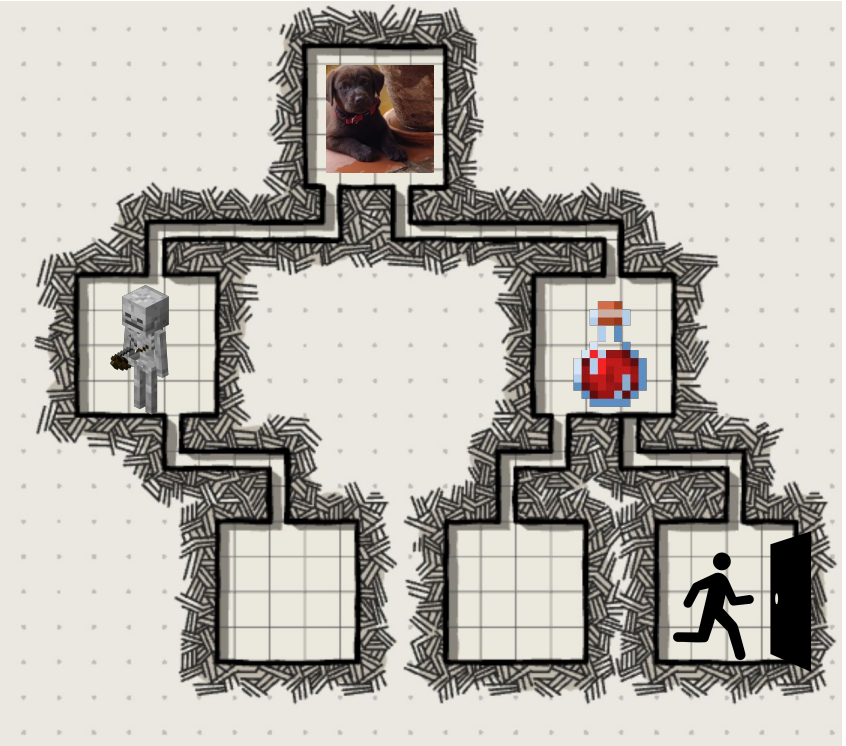


Refining our problem statement (2)

He’s also told us that there will be different kinds of things inside any given room.

A room might have:

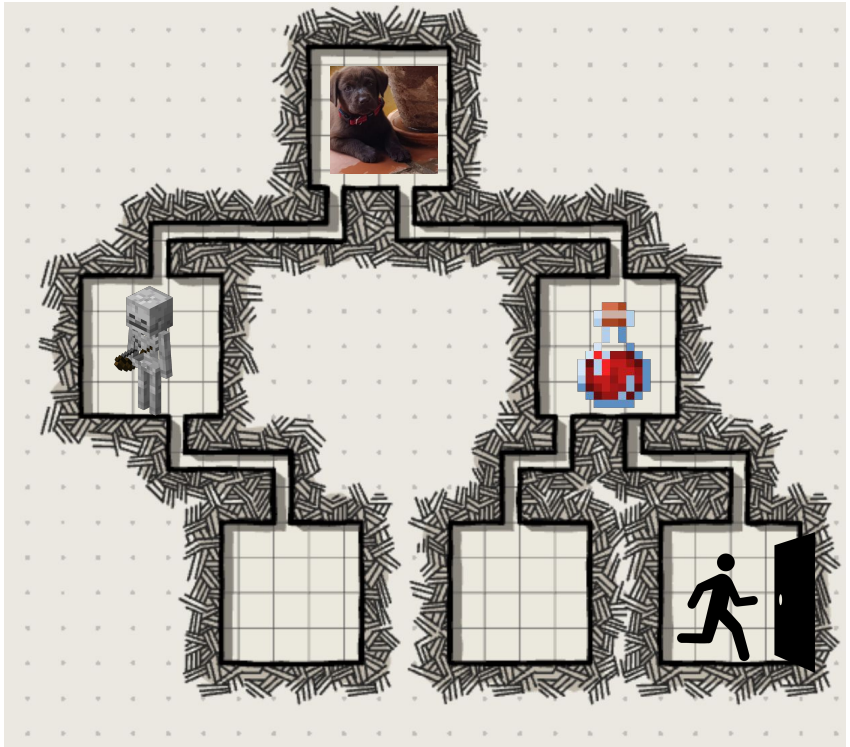
- Nothing inside (empty)
- A **monster** inside
- A **health potion** to help our dog recover from fighting monsters
- The **exit** which our dog needs to reach



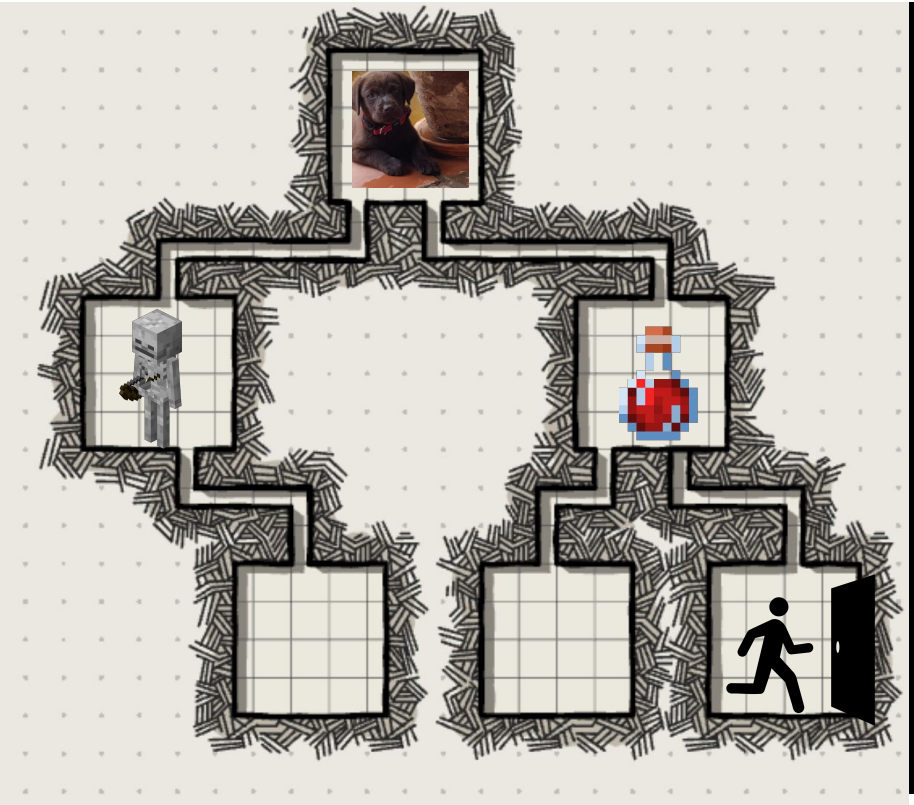
Refining our problem statement (3)

The goal is to **calculate the best possible outcome for the dog in the dungeon**, by taking the **best route** through it.

Generously, he isn't making us track which turns we took (he is going to evaluate it by hand, so he'll do that for us). **We only need to keep track of where our dog currently is, where it has visited, and its current attributes.**



Translating the problem statement into Haskell

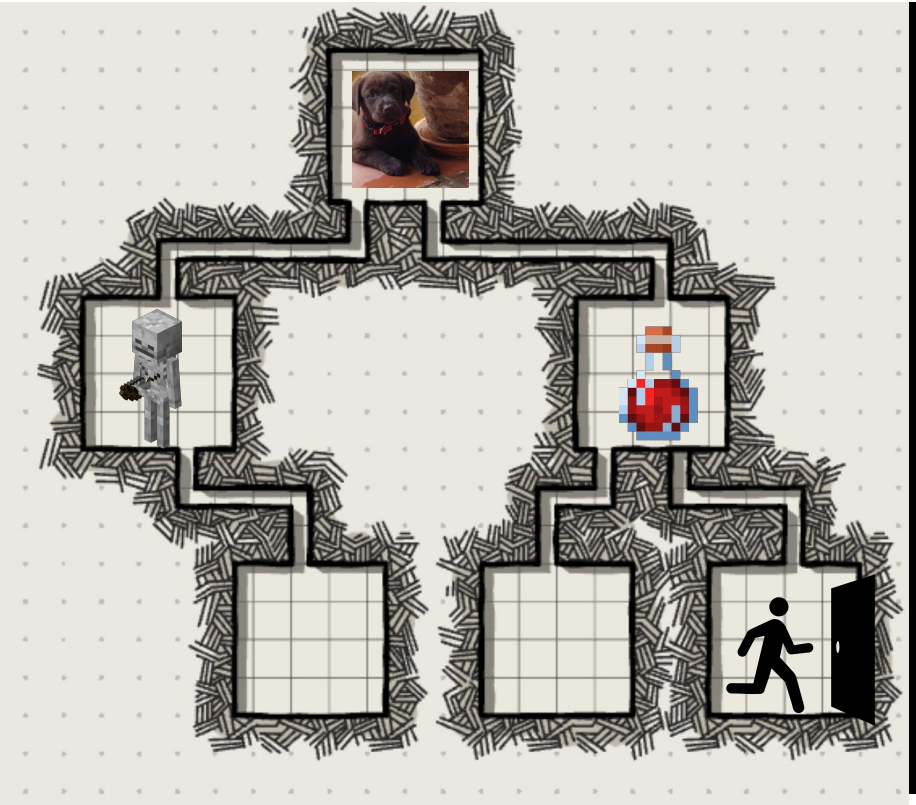


```haskell
-- A room has contents, and might branch
-- into 0, 1, or 2 rooms. The wizard promised
-- we won't ever have a cycle (rooms can't
-- connect to previous rooms).
-- Therefore, Room is a _tree_ datastructure

data Room = Room RoomContents
                 (Maybe Room) (Maybe Room)
                 deriving Show

data RoomContents = Fight Monster
                  | Heal Int
                  | Empty
                  | Escape
                  deriving Show
```

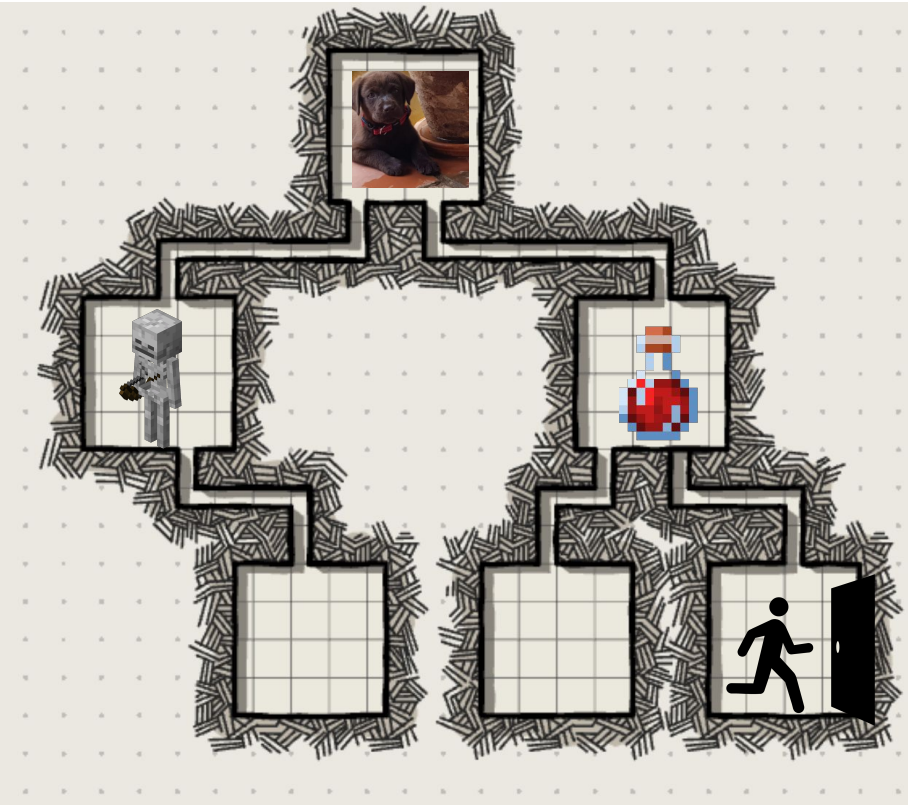
Translating the problem statement into Haskell (2)



```haskell
-- Our dog :)
data Dog = Dog
  {
    dhp  :: Int
  , ddmg :: Int
  } deriving Show

-- A monster! :o
data Monster = Monster
  {
    mhp  :: Int
  , mdmg :: Int
  } deriving Show
```

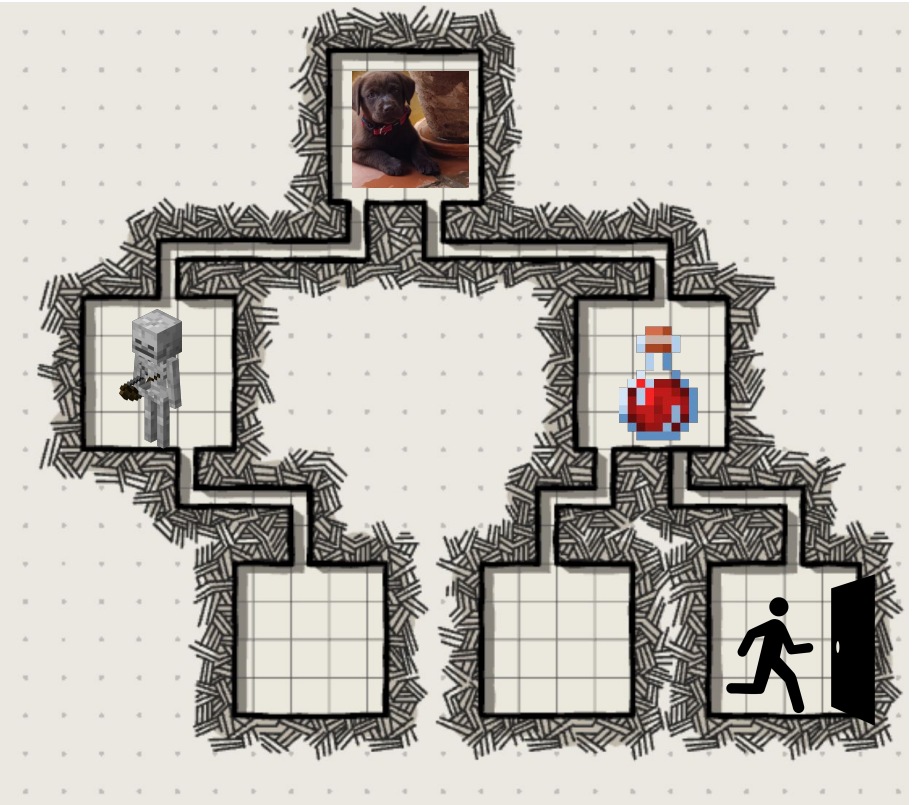
Translating the problem statement into Haskell (3)



```haskell
-- What the result of a dog entering a room is
-- They can either:
-- 1. Victory: escape
-- 2. Death: die
-- 3. Continue: be able to continue in the
dungeon; alive but not free yet.
data VisitResult = Victory
                 | Death
                 | Continue Dog
                 deriving Show

-- Our solution is the implementation of this:
-- Find the result of our dog visiting a room
AND its children rooms.
crawl :: Dog -> Room -> VisitResult
```

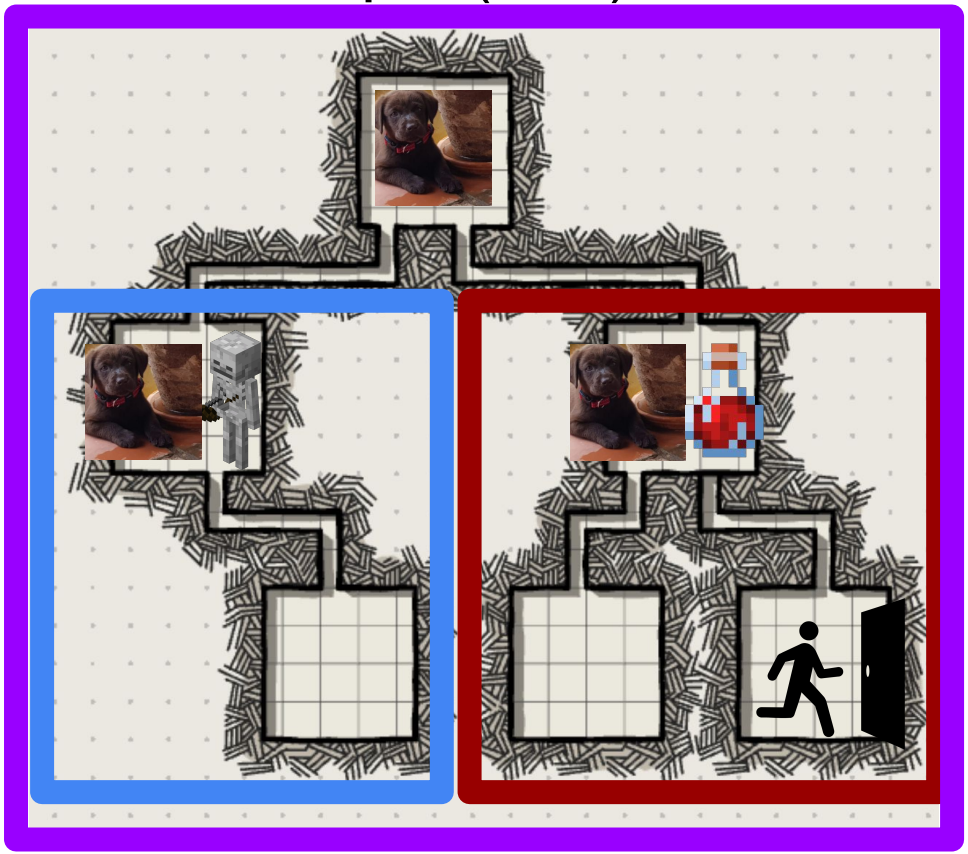
Solving the problem



We can solve this using a **divide-and-conquer approach.** That is, we have **subproblems** that we can solve, and **combining them gives us the optimal solution** (from TOA).

Here, our subproblem is *each subtree.* We combine them by checking which rooms to visit and taking the best ones only.

D&C example (root)

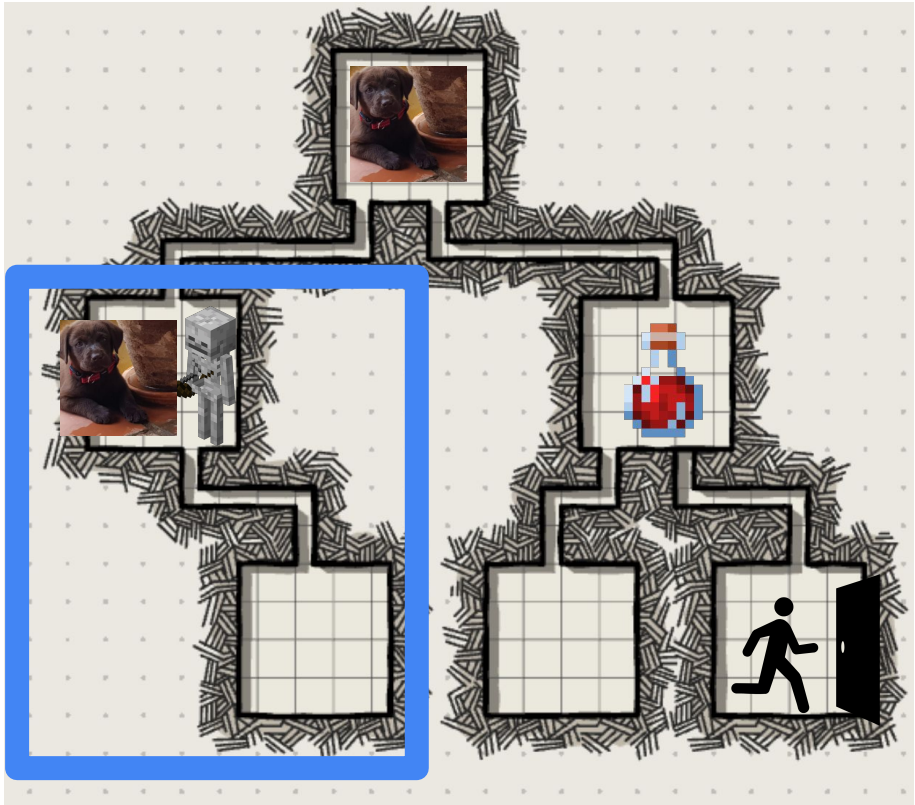


To solve the entire problem

… we need to check which of the two subtrees (left and right) is better.

We'll assume for the future that we start with `Dog { mhp = 10, ddmg = 5 }`

D&C example (root -> left)

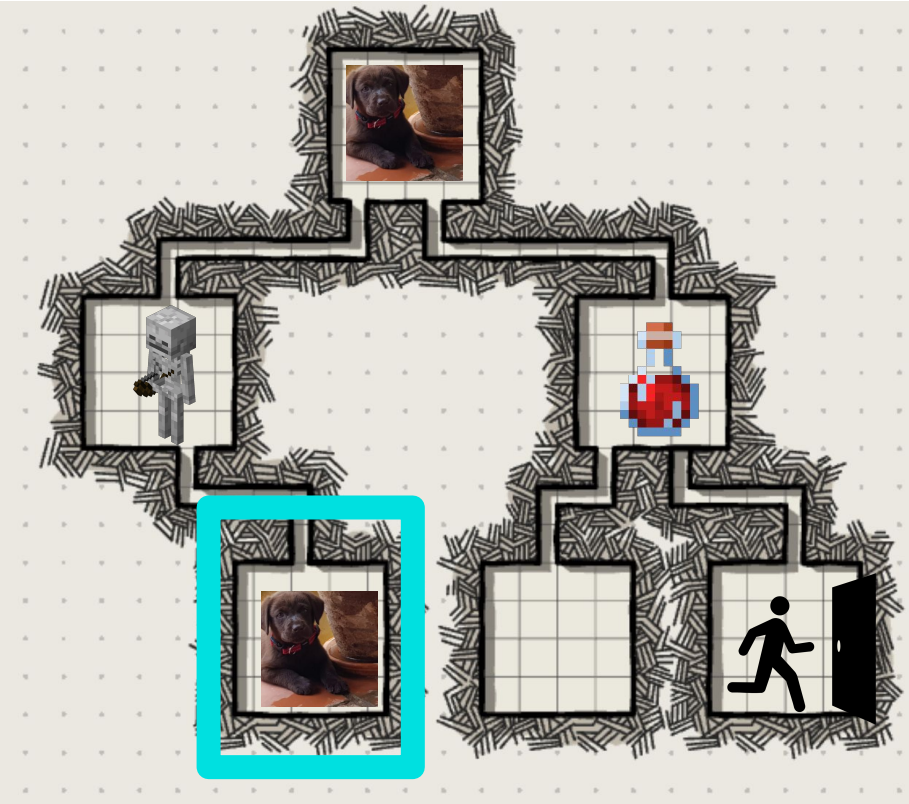


We first need to **figure out what the dog does here** - when it encounters a skeleton.

```
-- Find the result of our dog fighting a
monster
fight :: Dog -> Monster -> VisitResult
fight dog mon
  | dhp dog <= 0 = Death
  | mhp mon <= 0 = Continue dog
  | otherwise    = fight dog' mon'
    where
  dog' = (dog { dhp = (dhp dog) - (mdmg mon) })
  mon' = (mon { mhp = (mhp mon) - (ddmg dog) })
```

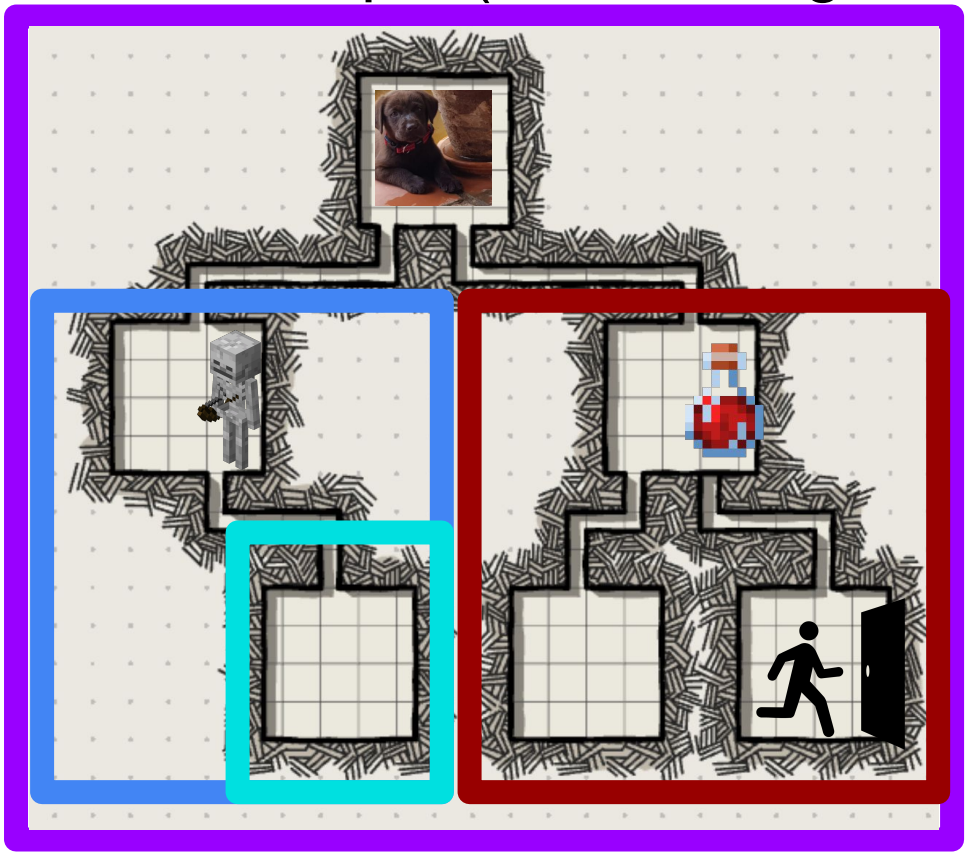
We'll say in this case we get `Continue Dog 5 5`

D&C example (root -> left -> right)



Now we need to check what happens in the next room… which is empty.
So, nothing happens.

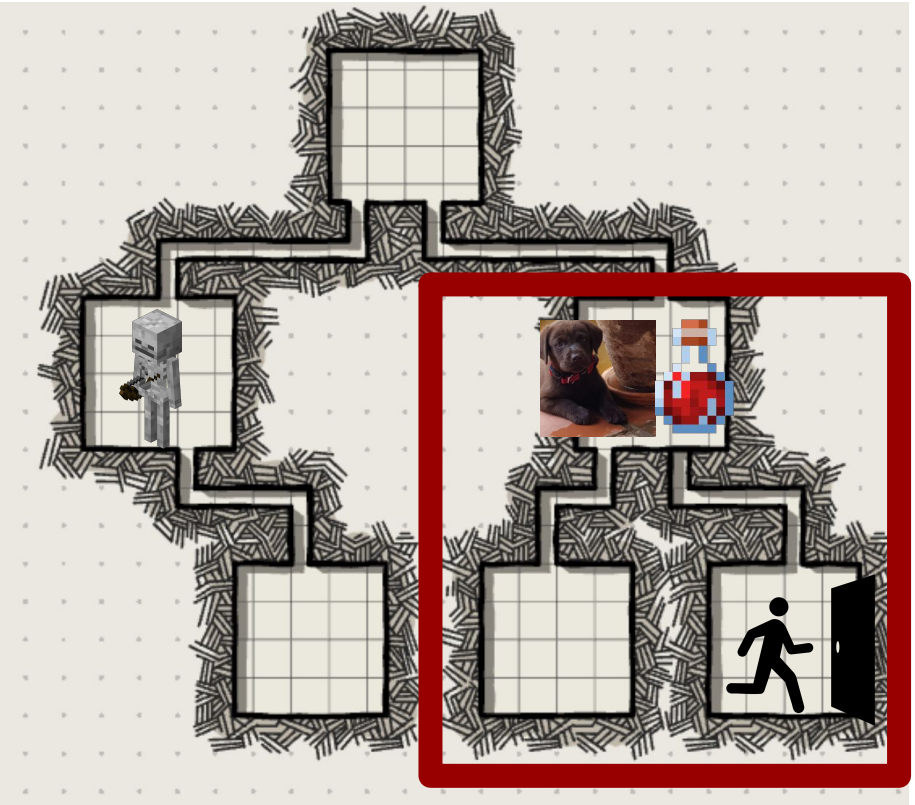
D&C example (root, having calc'd left path)



Propagating this back up to the top, we now know:

- The best solution for root -> left -> right is `Continue dog''`, where dog is the same as the previous room's dog…
- The best solution for root -> left is `Continue dog'`, where `dog'` is given by `fight dog skeleton`
- We also know that `dog'` will have lower health than dog.
- Therefore, our best solution is either to go right, or stay in the root room (slowly starving to death…)

D&C example (root -> right)

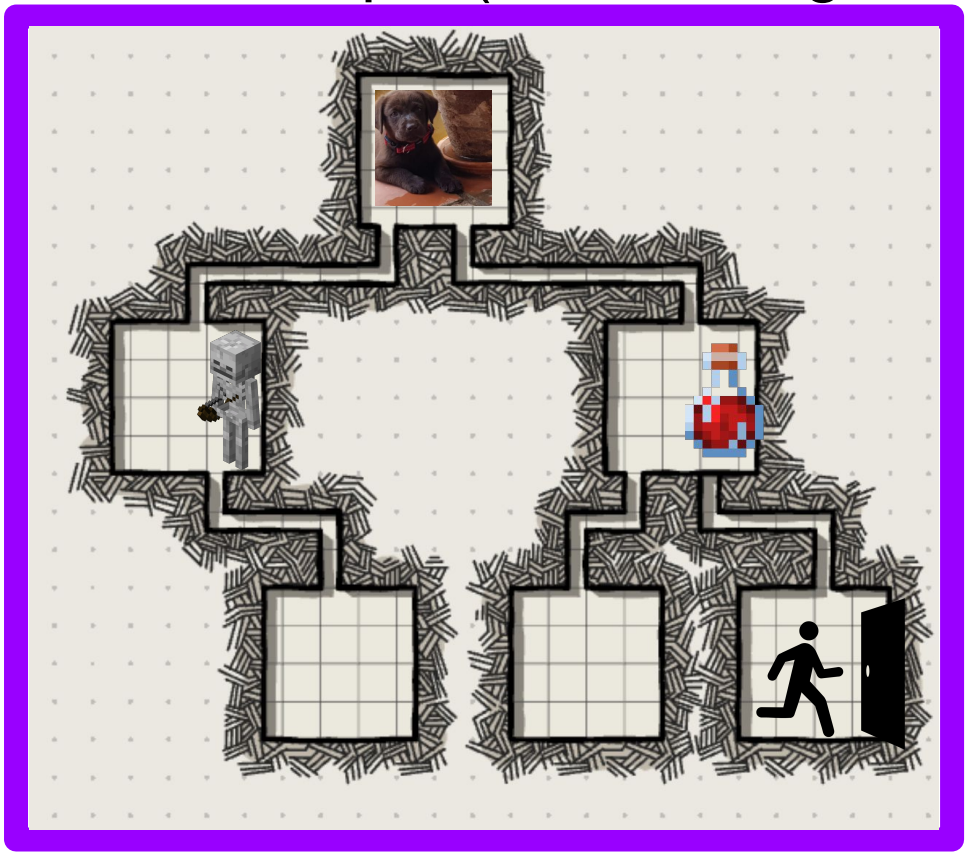


What would be the solution for this subtree?

We assume that we start with `Dog { dhp = 10, ddmg = 5 }` and the healing room is given by `Heal 10`.

1. `Continue (Dog 10 5)`
2. `Continue (Dog 20 5)`
3. `Continue (Dog 10 15)`
4. `Victory`

D&C example (root, having calc'd left + right paths)



So, our **overall** result for the **entire tree** would be `Victory`!

Recall that the possibilities we checked were:

1. Stay there = `Continue (Dog 10 5)`
2. Go left = `Continue (Dog 5 5)`
3. Go right = `Victory`

Comparing solutions (`VisitResult`s)

Recall that the possibilities we checked were:

1. Stay there = `Continue (Dog 10 5)`
2. Go left = `Continue (Dog 5 5)`
3. Go right = `Victory`

But how will we represent in *code* which one is best? We need a way to **compare & rank** different `VisitResults`.

```haskell
-- We define Ord here such that x <= y means that x is a _worse result_ than y for our dog
instance Ord VisitResult where
    a <= b
      -- == obviously implies <=, since <= is defined as < or ==
      | a == b = True      -- matches a <= b IF a == b.

      -- If both paths continue, the best one is when the dog has the most health
      | (Continue dog) <- a, (Continue dog') <- b = (dhp dog) <= (dhp dog')

      -- Now we've ruled out == from <=, we just need to implement < for the rest

      -- Death is worse than anything else, and anything is better than Death
      | Death <- a = True  -- matches Death <= any
      | Death <- b = False -- matches any <= Death

      -- Victory is better than anything else, and anything is worse than Victory
      | Victory <- b = True  -- matches any <= Victory
      | Victory <- a = False -- matches Victory <= any

      -- DIFFERENCE FROM THE LECTURE: I got rid of the otherwise case to make it
      -- easier to see what is happening here.
```

Ord requires an impl of Eq

```
-- Eq is defined like this.
-- This means to implement Ord we also need to implement Eq.
class Eq a => Ord a

-- Our ddmg shouldn't ever change, so we only compare HP here
-- And we won't compare dogs from different dungeons, so we don't
-- need to worry about dogs with different dmg values being compared
instance Eq VisitResult where
    Victory      == Victory      = True
    Death        == Death        = True
    (Continue a) == (Continue b) = (dhp a) == (dhp b)
    _ == _                       = False
```

Aha! You thought you'd be free from a quiz!

1. (2 points) What is a valid **type** to infer for `sum . (map (+2))`?
   - **a.** Num a => a -> a -> a
   - **b.** Num a => [a] -> a
   - **c.** Num a => (a -> a) -> [a] -> a
   - **d.** Num a => (a -> a) -> [a] -> a -> a
2. (1 point) A lambda is…
   - **a.** A function with only one argument
   - **b.** An anonymous function
   - **c.** A generic datatype
   - **d.** The inside of a list comprehension

Solving one room

I already showed you how we would solve a dog encountering a skeleton. But it can also encounter empty rooms, exit rooms, and health potion rooms. **We need a function that can check what happens for any given room.**

We will leave out the children of a room for now, and return to this later. It makes our code a bit neater.

```
-- Find the result of our dog visiting a
-- room. This is the CONQUER part of our
-- divide and conquer approach.
visitRoom :: Dog -> RoomContents -> VisitResult
visitRoom dog room = go room
    where
  go (Fight monster) = fight dog monster
  go (Heal hp)       = Continue dog { dhp = (dhp dog) + hp }
  go Escape          = Victory
  go Empty           = Continue dog
```

Solving the children of a room

```haskell
-- This is our DIVIDE part of our divide and conquer approach.
fork :: Dog -> Maybe Room -> Maybe Room -> VisitResult
fork dog leftRoom rightRoom = bestPath
    where
  -- The best path for the dog is the _highest scoring_ valid path.
  bestPath   = maximum validPaths

  -- Our possible valid paths.
  -- An _invalid_ path would be Nothing here, and represents e.g. going to the left room,
  -- if there _is_ no left room. catMaybes filters these out into paths that CAN be done.
  validPaths = catMaybes
                [leftFirst, rightFirst, leftOnly, rightOnly, Just $ Continue dog]

  -- Define the valid paths
  leftFirst  = fmap (uncurry $ crawlBoth dog) (both leftRoom rightRoom)
  rightFirst = fmap (uncurry $ crawlBoth dog) (both rightRoom leftRoom)
  leftOnly   = fmap (crawl dog) leftRoom
  rightOnly  = fmap (crawl dog) rightRoom
```

```haskell
-- Convert two maybes into one.
-- We get a Just out when both maybes are Just.
both :: Maybe a -> Maybe b -> Maybe (a, b)
both (Just a) (Just b) = Just (a, b)
both _        _        = Nothing

-- Find the result of our dog visiting the first room
-- and THEN the second room, in sequence.
crawlBoth :: Dog -> Room -> Room -> VisitResult
crawlBoth dog first second = sequenced
    where
  sequenced = case (crawl dog first) of
                Continue dog' -> crawl dog' second
                Victory       -> Victory
                Death         -> Death
```

Side note: fmap + uncurry

`fmap` is like `map` but more general. On a Maybe type, we'd have

```
fmap f (Just x) = Just (f x)
fmap _ _        = Nothing
```

So we use it to modify the value *inside* the Maybe, if it is present.

`uncurry` is useful to transform functions operating on two arguments (curried functions) to a function working on *one* argument of a pair.

```
uncurry f = \(a, b) -> f a b
```

```
(uncurry (+)) (1, 2) == (+) 1 2
```

Putting it all together

```
-- Find the result of our dog visiting a room AND its children rooms.
-- This essentially solves the subtree rooted at the given room.
-- Therefore, if we call it on the root, we solve the whole dungeon!
crawl :: Dog -> Room -> VisitResult
crawl dog (Room this left right) = soln
    where
  soln = case (visitRoom dog this) of
              Continue dog' -> fork dog' left right
              Victory       -> Victory
              Death         -> Death
```

What the prac will be like

You will be given the full example solution here (also available on Amathuba under Haskell - `BaseGame.hs`)

Each prac session has a different set of questions (all equivalent in difficulty).

Question 1 is a **very scaffolded** question. You can get a lot of partial marks here, **AS LONG AS YOUR CODE COMPILES.** It is incremental, so Question 1.2 builds on 1.1, and so on. **I recommend answering Question 1 by first creating a BLANK Q1Ans.hs file, and SLOWLY copying & modifying code from the provided skeleton** (the base game you're given).

What the prac will be like (2)

Question 2 is **not scaffolded**. It will most likely build off of question 1, but crucially **you probably won't get marks for code that simply compiles!**

You can *still* get a lot of partial marks here, **AS LONG AS YOUR CODE COMPILES.** You will get partial marks for a basic solution, and more and more marks for handling more and more difficult inputs. A perfect solution compiles + handles all cases.

QUESTION 2 SHOULD BE SUBMITTED IN A DIFFERENT FILE TO QUESTION 1. You can copy the code over to *start* on question 2, **BUT YOU CANNOT GO THE OTHER WAY.** I.e. Question 2 may be different enough that it will no longer solve Question 1.

Sample prac test

The wizard has stolen the class dog… but he's also stolen one of your pet rats! You should try fetch it also if possible. **If your dog would hurt at all in order to fetch the rat, then leave it behind** (it'll probably find a way out on its own eventually, since it can burrow…). This means means that the *net health* of the best path that gets the rat should be >= best path that doesn't get the rat.

Sample prac test Q1

1.1) Implement a type for Rat. Give it a member `ratName` of type `String`. You should be able to get the data from `Rat` using the `ratName :: Rat -> String` function.

1.2) Add a member `rat` to the `Dog` type. Make sure it is of type `Maybe Rat`.

1.3) Add a variant to the `RoomContents` type for a room with the rat in it. Call it `Cage`. It should have a value of type `Rat`.

1.4) Modify the `Eq` implementation for `Dog` to check the two `Dogs` given both have rats. If they don't, then the dogs won't be equal.

1.5) Modify the Ord implementation for Dog such that Dogs without a rat are less than dogs with a rat, *IF* their healths are equal. So `(Dog 10 2 Nothing) <= (Dog 10 2 (Just Rat "Jeremy"))` , BUT `(Dog 4 2 (Just Rat "Jeremy")) <= (Dog 10 2 Nothing)`

Sample prac test Q2

2) OH NO! *ALL* YOUR PET RATS HAVE BEEN TAKEN! (not just the one)

Implement the full solution for dungeons that can have *multiple* rats in them. Still only one per room, though.

You'll need to:

i) modify Dog to hold multiple rats (call the member `rats`)
ii) Implement a function to get how many rats a dog has. It should be called `ratCount`, take a `Dog`, and return an `Int`.
iii) modify `VisitResult`'s `Victory` variant to hold a `Dog`, so that we can see how many rats the `Dog` fetched.
iv) modify visitRoom to pick up a rat when in a room with a rat
v) modify the Eq and Ord implementations for Dog so that the best solution is one that picks up the *most* rats. This time, one rat is worth 2 health points of the dog. So, a solution involving a dog with 10 HP and 5 rats is **better** than a rat with 19 HP and no rats. Still avoid death, though.

Exercise

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

You will notice that `crawlBoth` and `crawl` have very similar `case` statements in their bodies. Define a function to unify these `case` statements: `flatmapContinue :: (Dog -> VictoryResult) -> VictoryResult -> VictoryResult`

It should be implemented such that we can implement `crawlBoth` with this body:

```
flatmapContinue (\dog' -> crawl dog' second) $ crawl dog first
```

And `crawl` with this body:

```
flatmapContinue (dog' -> fork dog' left right) $ visitRoom dog this
```

```haskell
-- Find the result of our dog visiting a room AND its children rooms.
-- This essentially solves the subtree rooted at the given room.
crawl :: Dog -> Room -> VisitResult
crawl dog (Room this left right) = case (visitRoom dog this) of
    Continue dog' -> fork dog' left right
    Victory       -> Victory
    Death         -> Death
```

```haskell
-- Find the result of our dog visiting the first room and THEN the second room, in sequence.
crawlBoth :: Dog -> Room -> Room -> VisitResult
crawlBoth dog first second = case (crawl dog first) of
    Continue dog' -> crawl dog' second
    Victory       -> Victory
    Death         -> Death
```